

Abstract.

Resumo.

1. Introduction

A detecção de tráfego malicioso é um problema central em cibersegurança. Segundo o *Verizon Data Breach Investigations Report* (DBIR) de 2026 [Verizon Business 2026], os maiores ataques distribuídos de negação de serviço (DDoS) apresentaram crescimento de 198% no volume de bits por segundo; a exploração de vulnerabilidades tornou-se o principal vetor inicial de invasão, representando 31% das violações registradas e crescimento de 55% em relação ao período anterior; e o uso de inteligência artificial por agentes maliciosos acelera a descoberta de falhas, automatiza a exploração de vulnerabilidades e potencializa etapas dos ataques cibernéticos, aumentando a complexidade dos mecanismos tradicionais de defesa.

Nesse cenário, os sistemas de detecção de intrusão (IDS) baseados em assinatura dependem de padrões conhecidos e bem definidos, mas apresentam limitações claras frente a ataques novos ou ofuscados, o que motiva abordagens estatísticas e de aprendizado de máquina (AM) capazes de generalizar para padrões não vistos. Além disso, abordagens baseadas em AM enfrentam desafios adicionais, como a escassez de *datasets* de referência publicamente disponíveis e o envelhecimento rápido desses *benchmarks* à medida que as tecnologias de rede evoluem [He et al. 2023].

Nesse contexto, este trabalho treina modelos de aprendizado de máquina para detecção binária de tráfego malicioso, a fim de obter classificadores capazes de distinguir tráfego legítimo e malicioso e de generalizar para ataques novos ou ofuscados, superando as limitações das abordagens por assinatura e mitigando os efeitos do envelhecimento de *datasets*. Para isso, utiliza o conjunto de dados CICIDS-2017 [Sharafaldin et al. 2018], amplamente adotado como *benchmark* de detecção de intrusão, com 14 tipos de ataques rotulados e mais de 2 milhões de amostras de fluxos de rede. Este trabalho também explora a interpretabilidade dos modelos treinados e a reprodutibilidade dos experimentos propostos.

As principais contribuições deste trabalho são: (i) comparação sistemática de cinco classificadores para detecção binária no CICIDS-2017; (ii) análise de interpretabilidade do melhor modelo (SHAP, importância de atributos); (iii) pipeline reproduzível (código/Docker).

O restante deste artigo está organizado da seguinte forma. A Seção 2 revisa trabalhos relacionados à detecção de intrusão com aprendizado de máquina. A Seção 3 descreve a metodologia, incluindo o *dataset*, o pré-processamento, o *pipeline* de treinamento, as métricas de avaliação, a interpretabilidade dos modelos e a reprodutibilidade dos experimentos. A Seção 4 apresenta os experimentos e os resultados obtidos. A Seção 5 discute os achados e limitações do estudo. Por fim, a Seção 6 apresenta as conclusões e perspectivas de trabalhos futuros.

2. Related Work

A literatura sobre detecção de intrusão baseada em aprendizado de máquina (AM) consolidou o uso de *benchmarks* públicos de tráfego rotulado para comparar classificadores e re-

latórios de desempenho. Entre esses conjuntos, o CICIDS-2017 [Sharafaldin et al. 2018] tornou-se referência recorrente por reproduzir fluxos benignos e maliciosos com dezenas de atributos derivados de fluxos de rede. Nesta seção, revisamos três trabalhos representativos: dois estudos empíricos no CICIDS-2017 e um *survey* sobre AM adversarial em NIDS.

Jairu e Mailewa [Jairu and Mailewa 2022] propõem uma abordagem supervisionada de inteligência artificial para descoberta de anomalias de rede no CICIDS-2017. O estudo enfatiza a insuficiência de IDS baseados apenas em assinaturas e avalia múltiplos classificadores supervisionados sobre características de fluxo, com foco em identificar padrões de tráfego malicioso frente a tráfego legítimo. Os autores discutem o papel do pré-processamento e da seleção de atributos para elevar a acurácia na detecção de ataques conhecidos no *dataset*, posicionando o AM como alternativa prática à detecção puramente baseada em regras.

Maseer *et al.* [Maseer et al. 2021] conduzem um *benchmark* sistemático de algoritmos de AM para IDS baseados em anomalias no CICIDS2017. O artigo compara dez técnicas supervisionadas e não supervisionadas—incluindo árvores de decisão, k -NN, Naive Bayes, floresta aleatória, SVM e redes neurais—em dezenas de configurações de modelos, reportando acurácia, precisão, *recall*, F1 e tempo de treinamento. Os resultados indicam que modelos como k -NN, árvore de decisão e Naive Bayes obtêm desempenho competitivo em classes de ataques específicas, mas o desbalanceamento e a diversidade de ataques no CICIDS2017 exigem critérios de avaliação cuidadosos.

Em perspectiva mais ampla, He *et al.* [He et al. 2023] apresentam um *survey* sobre aprendizado de máquina adversarial aplicado a sistemas de detecção de intrusão em rede (NIDS). Os autores organizam taxonomias de NIDS baseados em aprendizado profundo, formulam ataques adversariais voltados a evasão de modelos e revisam mecanismos de defesa. O levantamento destaca desafios estruturais da área—como escassez de *datasets* públicos confiáveis, obsolescência rápida de *benchmarks* e limitações de interpretabilidade em modelos de alto desempenho—que motivam avaliações reproduzíveis e análises explicáveis, e não apenas ganhos marginais de acurácia.

Posicionamento deste trabalho. Os estudos de Jairu e Mailewa [Jairu and Mailewa 2022] e de Maseer *et al.* [Maseer et al. 2021] concentram-se principalmente no desempenho preditivo de classificadores no CICIDS-2017, em geral em cenários multiclasse ou fortemente orientados a métricas globais. O *survey* de He *et al.* [He et al. 2023] sistematiza riscos e lacunas de pesquisa, mas não propõe um pipeline experimental único com foco em interpretabilidade. Este trabalho diferencia-se por: (i) adotar detecção binária (tráfego legítimo *versus* malicioso) com cinco classificadores clássicos; (ii) analisar interpretabilidade do melhor modelo (por exemplo, SHAP e medidas de importância de atributos); e (iii) documentar a reprodutibilidade dos experimentos (código e ambiente containerizado). Assim, complementamos *benchmarks* existentes no CICIDS-2017 com uma visão orientada à explicabilidade e à replicação, em linha com as recomendações de pesquisa apontadas por He *et al.* [He et al. 2023].

3. Methodology

3.1. Dataset

O *dataset* utilizado é o CICIDS-2017 [Sharafaldin et al. 2018], produzido pelo Canadian Institute for Cybersecurity. O conjunto compreende 14 tipos de ataques, em um cenário de classificação multiclasse nos rótulos originais; neste trabalho, os fluxos são agregados para detecção binária (tráfego legítimo *versus* malicioso).

A distribuição agregada dos rótulos é severamente desbalanceada: 80,3% de fluxos BENIGN e 19,7% ATTACK após unificar os tipos de ataque (Figura 1). Entre as classes de ataque individuais, a disparidade é ainda maior; categorias raras, como Heartbleed (7 amostras) e Web_Attack_Sql_Injection (21 amostras), tornam inviável um treinamento multiclasse estável no escopo deste estudo (Figura 2).

3.1.1. Formulação binária

Dada a dificuldade de aprender classes com menos de 40 amostras no prazo do projeto, agrupamos todos os ataques na classe ATTACK e definimos a tarefa como classificação binária (BENIGN *versus* ATTACK). Reconhecemos que essa escolha reduz a granularidade por tipo de ataque; essa limitação é discutida na Seção 5.

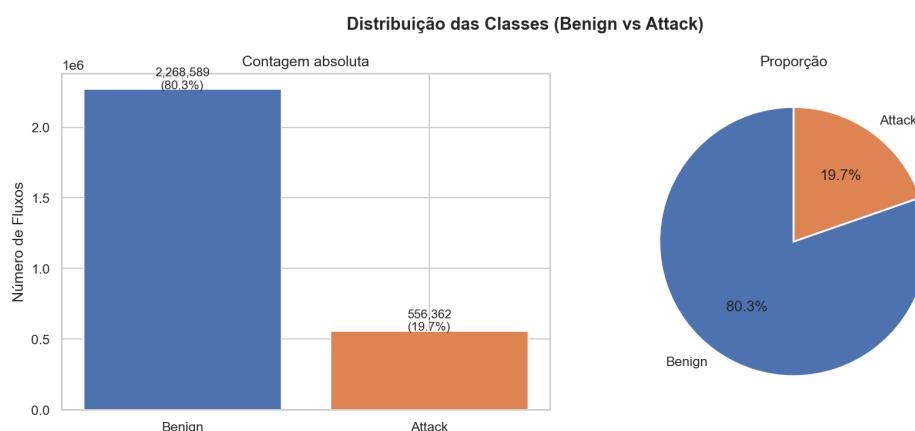


Figura 1. Distribuição agregada BENIGN *versus* ATTACK no CICIDS-2017.

3.2. Data and preprocessing

Unificamos os oito arquivos CSV do CICIDS-2017 e aplicamos um *pipeline* de limpeza e redução de dimensionalidade, guiado por análise exploratória (EDA), antes do treinamento dos classificadores.

Os problemas tratados foram:

- **Duplicatas:** 255.234 linhas repetidas (9,03% do total), removidas com base nas colunas de fluxo (excluindo metadados de origem do arquivo).
- **Valores infinitos:** nas colunas *Flow Bytes/s* e *Flow Packets/s*, gerados por divisão por zero; substituídos por NaN e descartados na limpeza subsequente.

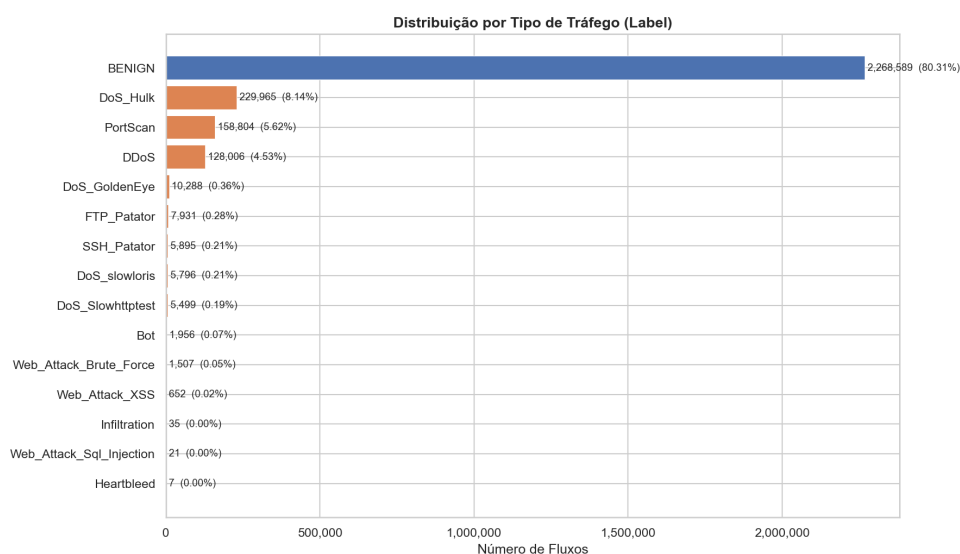


Figura 2. Distribuição por tipo de ataque no CICIDS-2017.

- **Variância zero:** oito atributos constantes em todo o conjunto (*_Avg_Bulk_*, Bwd PSH Flags, Bwd URG Flags), eliminados por não contribuírem para a discriminação.
- **Coluna redundante:** Fwd Header Length.1, idêntica a Fwd Header Length, removida.
- **Alta correlação:** remoção iterativa de atributos com $|r| \geq 0,95$ (Pearson), resultando em 23 *features* redundantes adicionais descartadas.
- **Outliers:** tratamento pelo método do intervalo interquartil (IQR), com limitação suave (*clipping*) nos extremos das variáveis numéricas, conforme a distribuição observada na EDA (Figura 3).

Após a agregação binária dos rótulos, a matriz final contém **47 atributos numéricos** e **2.824.951 fluxos**.

3.3. Training pipeline

O treinamento foi implementado em Python 3.11 com *scikit-learn*, em cinco etapas, com `random_state=42`, `class_weight=balanced` (sem SMOTE) e métricas sensíveis ao desbalanceamento (Seção 3.4).

Algoritmos avaliados. Comparamos cinco classificadores clássicos, escolhidos por interpretabilidade, custo computacional e uso recorrente em *benchmarks* de IDS: Regressão Logística (RL), Random Forest (RF), Árvore de Decisão (AD), Hist Gradient Boosting (HGB) e Naive Bayes Gaussiano (NB). A RL é treinada em um Pipeline com `StandardScaler` ajustado dentro de cada fold de validação, prevenindo *data leakage* do conjunto de validação no escalonamento; árvores e ensemble baseados em árvores (RF, AD, HGB) operam diretamente sobre os atributos originais.

Divisão dos dados. Aplicamos *holdout* estratificado 80/20 sobre a matriz final de fluxos, preservando a proporção BENIGN/ATTACK em treino e teste. O conjunto de teste permaneceu isolado de todas as etapas subsequentes—validação cruzada, testes

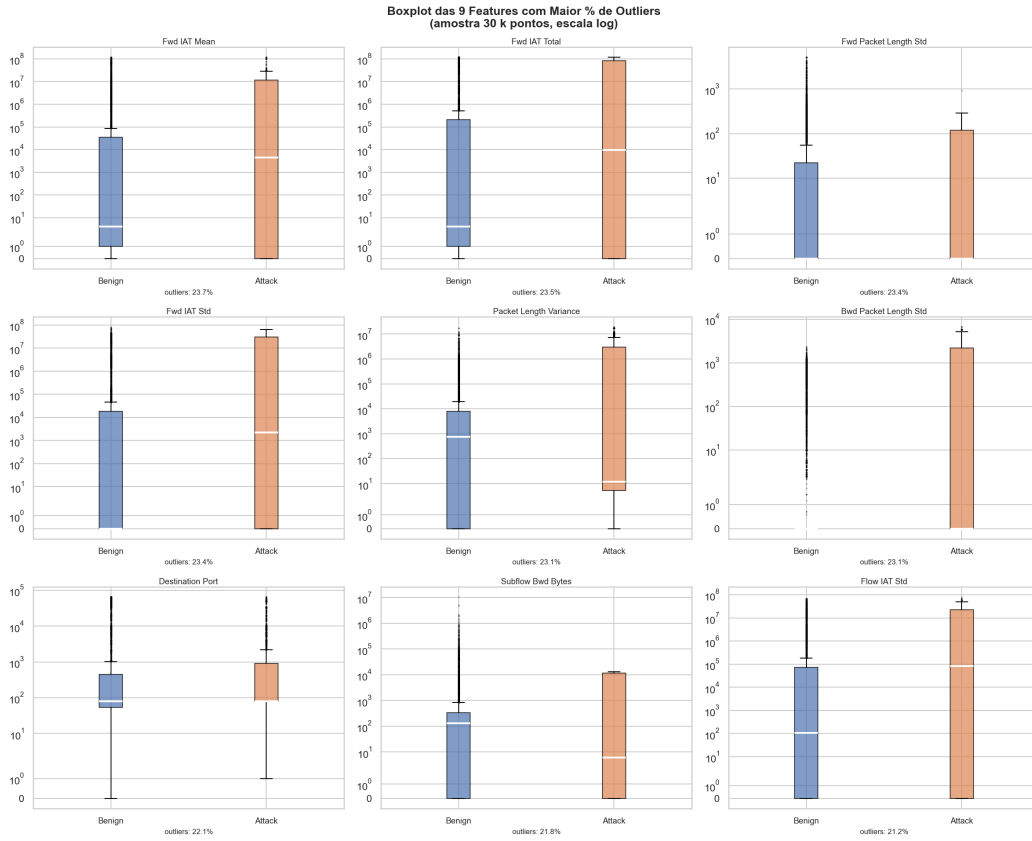


Figura 3. Boxplots dos atributos com maior incidência de outliers (IQR).

estatísticos e otimização de hiperparâmetros—e foi utilizado apenas na avaliação final reportada.

Experimentos.

1. **Spot-checking:** treino dos cinco modelos com hiperparâmetros padrão da biblioteca (por exemplo, RL com `max_iter=1000`, RF com `n_estimators=100`) sobre o conjunto de treino e avaliação no conjunto de teste, para triagem inicial.
2. **Validação cruzada:** os mesmos modelos foram reavaliados por *Repeated Stratified K-Fold* com $k = 10$ folds e três repetições (30 avaliações por modelo), exclusivamente no conjunto de treino, reportando média e desvio padrão de F1, precisão, *recall* e ROC-AUC (classe positiva ATTACK).
3. **Testes estatísticos:** teste de Friedman como procedimento *omnibus* sobre os 30 escores por modelo, seguido de Wilcoxon pareado com correção de Bonferroni entre pares de classificadores.
4. **Otimização de hiperparâmetros:** para os dois modelos mais promissores nas etapas anteriores (RF e HGB), *RandomizedSearchCV* com 20 combinações amostradas e *StratifiedKFold* interno com cinco folds; por viabilidade computacional, a busca usou uma amostra estratificada de 20% do treino, e o melhor modelo foi retreinado no treino completo.
5. **Avaliação final:** modelo otimizado avaliado no *holdout* de teste reservado.

3.4. Metrics and Evaluation

Avaliamos os classificadores com quatro métricas calculadas no *scikit-learn*, sempre com classe positiva `Attack` (tráfego malicioso): precisão (fração de alertas corretos entre os previstos como ataque), *recall* (fração de ataques reais detectados), F1 (média harmônica entre precisão e *recall*) e ROC-AUC (capacidade de separar as classes ao variar o limiar de decisão). Também reportamos acurácia (*accuracy*) como referência, mas não a usamos para ranquear modelos.

As classes previstas vêm de `predict()`, que no *scikit-learn* aplica limiar 0,5 sobre $P(\text{Attack})$ quando o modelo fornece probabilidades. No *spot-checking* e na avaliação final, precisão, *recall* e F1 usam `pos_label='Attack'` com rótulos categóricos `Benign/Attack`; o ROC-AUC usa `predict_proba` da classe `Attack` (métrica independente de limiar). Na validação cruzada, codificamos y como binário (1 = `Attack`, 0 = `Benign`) e aplicamos os *scorers* `f1`, `precision`, `recall` e `roc_auc`: os três primeiros medem a classe positiva codificada como 1 (comportamento binário focado em `Attack`, sem `average='macro'` ou `'weighted'`).

Adotamos o F1 da classe `Attack` como critério principal de comparação e seleção dos modelos. Com cerca de 80% de fluxos `Benign` e 20% `Attack`, um classificador que prevê sempre tráfego legítimo obtém acurácia elevada, porém F1 nulo para ataques—inadequado para detecção de intrusão, em que falhar em detectar `Attack` é o erro mais grave. O F1 equilibra precisão e *recall*: alta precisão reduz falsos alarmes; alto *recall* aumenta a cobertura de ataques reais.

No *spot-checking*, calculamos as cinco métricas no conjunto de teste reservado, geramos matriz de confusão e *report* de classificação por modelo, e ordenamos os resultados pelo F1 de `Attack`. Na validação cruzada (*Repeated Stratified K-Fold*, 30 avaliações por algoritmo), registramos média e desvio padrão de F1, precisão, *recall* e ROC-AUC apenas no conjunto de treino; esses escores alimentaram os testes de Friedman e Wilcoxon. Após a otimização de hiperparâmetros (RF e HGB), a avaliação final no *holdout* de teste repetiu o mesmo conjunto de métricas para estimar o desempenho de generalização.

3.5. Interpretability

Além do desempenho preditivo, analisamos *por que* o classificador separa tráfego legítimo e malicioso—requisito central em IDS, em que operadores precisam auditar alertas e validar se o modelo aprendeu sinais de rede plausíveis, e não artefatos do pré-processamento. Tomamos como referência o **Random Forest otimizado** (`rf_opt`), re-treinado no conjunto de treino completo após *RandomizedSearchCV*: no *holdout*, obteve o maior F1 da classe `Attack` entre os modelos ajustados (RF e HGB), além de ser naturalmente interpretável por árvores e compatível com explicadores exatos para florestas.

Importância global (Gini). Extraímos `feature_importances_` do RF, que agrega, em todas as árvores, a redução média de impureza de Gini por atributo. Trata-se de um ranqueamento global rápido, útil para triagem de variáveis, embora possa favorecer atributos com muitos valores distintos; por isso o complementamos com SHAP. As cinco variáveis mais importantes foram `Destination Port`, `Average Packet Size`, `Bwd Packet Length Min`, `Init_Win_bytes_backward` e `Bwd Packet Length Std` (Figura 4)—atributos ligados a portas, tamanhos de pacote e janelas TCP, coerentes com a natureza fluxo-a-fluxo do CICIDS-2017.

SHAP. Aplicamos SHAP (*SHapley Additive exPlanations*) [Lundberg and Lee 2017] com TreeExplainer, que calcula contribuições exatas para modelos baseados em árvores. Sobre uma amostra aleatória de 5.000 fluxos do conjunto de teste (`random_state=42`), obtivemos valores SHAP para a classe Attack: positivos empurram a predição para ataque e negativos para Benign. O *beeswarm plot* (Figura 5) resume o efeito de cada atributo no conjunto amostrado; o ranqueamento por $|SHAP|$ médio recuperou o mesmo núcleo de variáveis do Gini (porta de destino e estatísticas de pacotes/janelas no sentido *backward*). Para as cinco variáveis de maior $|SHAP|$ médio, geramos *dependence plots* (Figura 6), que mostram como o valor de um atributo altera a contribuição marginal e revelam não linearidades e interações com outras *features*. A concordância entre Gini e SHAP reforça que o modelo não depende de um único atributo espúrio, mas de um conjunto estável de descritores de fluxo.

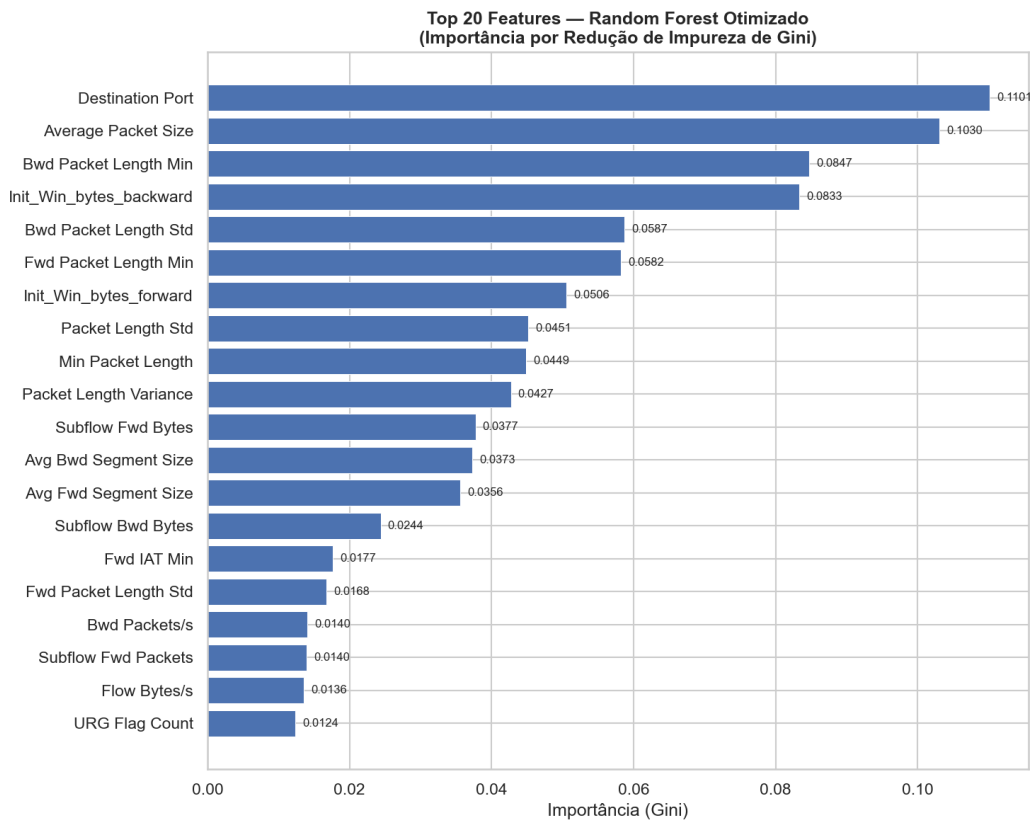


Figura 4. Top-20 atributos por importância de Gini no RF otimizado.

3.6. Reproducibility

Explicar como reproduzir o experimento. Docker e jupyter notebook.

4. Experiments and Results

Rodamos os x algoritmos, cada um levou x tempos e obtivemos x resultados.

5. Discussion

É possível perceber que tal modelo é o mais eficiente em detectar anomalias com base nesse dataset. É possível perceber que chegamos a tal de nível de interpretabilidade desse modelo. É possível perceber que existem caminhos em aberto que podem ser explorados para melhorar o desempenho do modelo, sobretudo avaliando o cenário de ataques específicos e não tão frequentes. É possível explorar como LLM's seriam capazes de aprender a detectar anomalias com base nesse dataset. É possível explorar como LLM's seriam capazes de aprender a gerar tráfego malicioso porém com comportamento que pareça ser legítimo para o modelo de detecção de anomalias. E como isso poderia ser usado para treinar um modelo de detecção de anomalias mais robusto ainda.

6. Conclusion

Tentamos encontrar o melhor modelo para detectar anomalias com base nesse dataset. Encontramos X

Tentamos fazer esse modelo ser interpretável Conseguimos Y

Entendemos que o problema é complexo e que ainda temos muito o que aprender sobre detecção de anomalias.

Referências

- He, K., Kim, D. D., and Asghar, M. R. (2023). Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 25(1):538–566.
- Jairu, P. and Mailewa, A. B. (2022). Network anomaly uncovering on CICIDS-2017 dataset: A supervised artificial intelligence approach. In *Proceedings of the 2022 IEEE International Conference on Electro Information Technology (eIT)*, pages 606–615.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774.
- Maseer, Z. K., Yusof, R., Bahaman, N., Mostafa, S. A., and Foozy, C. F. M. (2021). Benchmarking of machine learning for anomaly based intrusion detection systems in the CICIDS2017 dataset. *IEEE Access*, 9:124358–124372.
- Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116.
- Verizon Business (2026). 2026 data breach investigations report. Technical report, Verizon Business. Acesso em: 24 maio 2026.

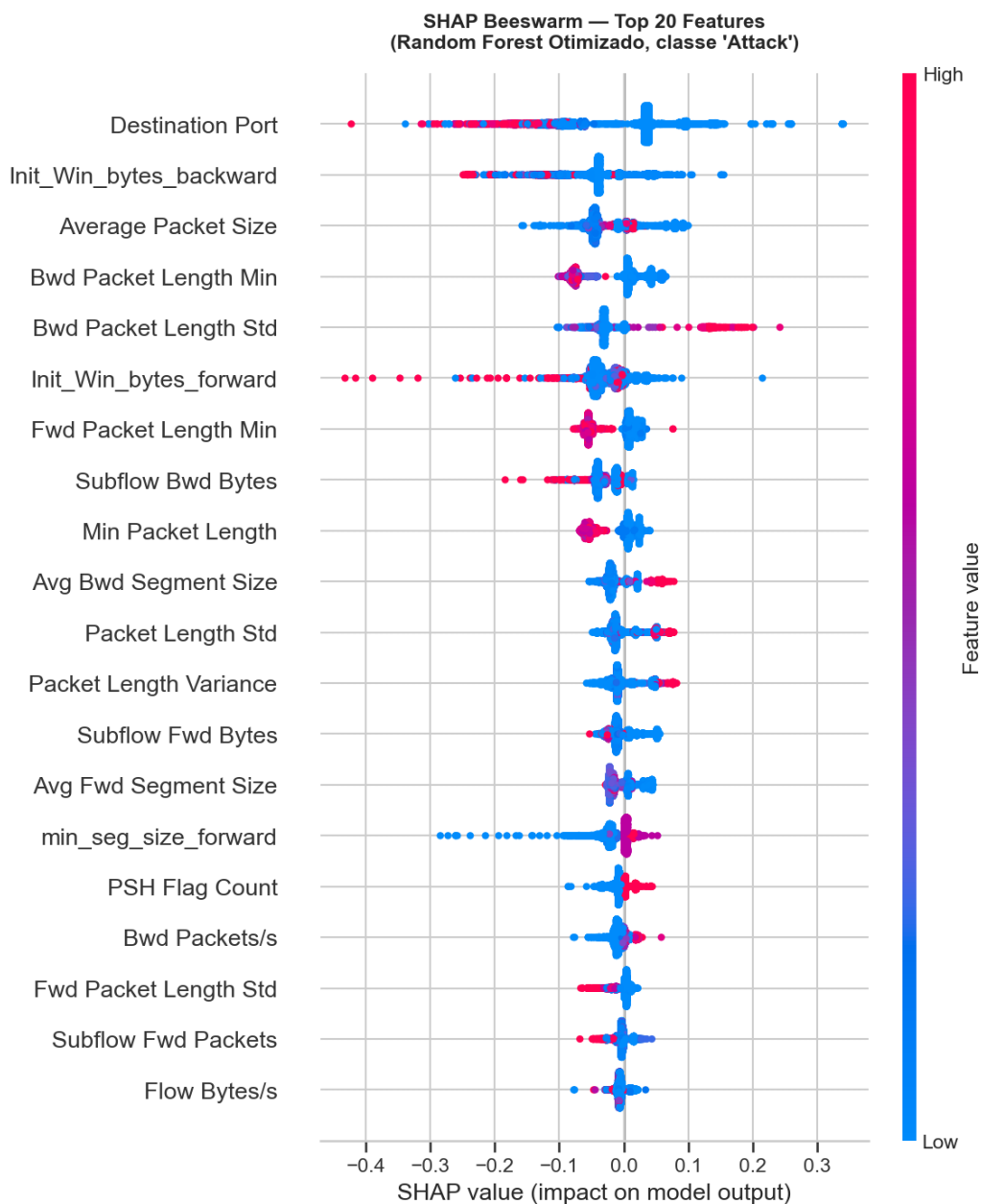


Figura 5. Resumo SHAP (beeswarm) para a classe Attack ($n = 5.000$ fluxos de teste).

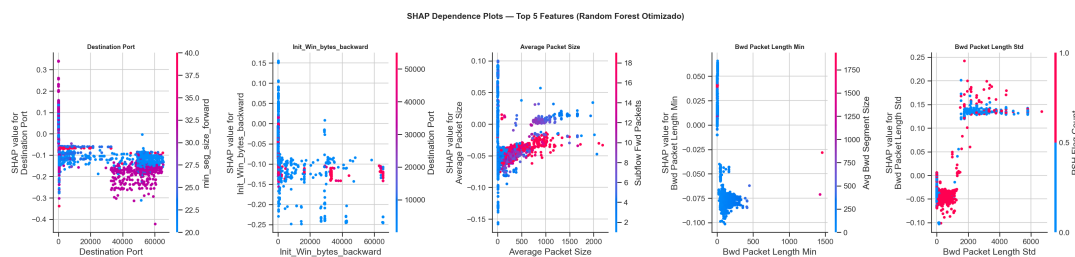


Figura 6. Dependence plots SHAP das cinco variáveis com maior $|SHAP|$ médio.